

Variabel, Konstanta & Operator

Memahami cara menyimpan data dan mengolahnya menggunakan berbagai jenis operator dalam C++

■ VARIABEL

- Deklarasi & Inisialisasi
- Aturan Penamaan
- Scope Variabel
- Auto & Type Inference

■ KONSTANTA

- const & constexpr
- #define Preprocessor
- Enum & Enum Class
- Perbandingan Ketiganya

■■ OPERATOR

- Aritmatika & Assignment
- Relasional & Logika
- Bitwise & Shift
- Ternary & sizeof

01 Variabel — Deklarasi, Penamaan & Scope

02 Konstanta — const, #define, constexpr

03 Enumerasi (enum & enum class)

04 Operator Aritmatika

05 Operator Assignment (Penugasan)

06 Operator Relasional & Logika

07 Operator Bitwise & Shift

08 Operator Ternary, sizeof & Precedence

BAB 1 — Variabel

1.1 Pengertian & Sintaks Deklarasi

Variabel adalah wadah bernama di memori untuk menyimpan data yang nilainya dapat berubah. Setiap variabel memiliki **tipe data**, **nama**, dan **nilai**.

```
// Sintaks: tipe_data nama_variabel = nilai_awal;

int    umur      = 20;
float  berat      = 65.5f;
double tinggi     = 175.3;
char   inisial    = 'A';
bool   lulus      = true;
string nama       = "Andi";

// C++11: uniform initialization (direkomendasikan)
int    skor       {95};
int    nol        {};           // otomatis 0

// C++11: auto — tipe disimpulkan compiler
auto   nilai      = 88;         // int
auto   pi         = 3.14159;    // double
auto   huruf      = 'B';        // char
```

1.2 Aturan Penamaan Variabel

Aturan	Contoh Benar	Contoh Salah
Mulai huruf atau underscore	<code>umur</code> , <code>_data</code>	<code>1nilai</code>
Hanya huruf, angka, underscore	<code>total_1</code>	<code>total#1</code>
Tidak boleh keyword C++	<code>myInt</code>	<code>int</code> , <code>for</code> , <code>if</code>
Case-sensitive	<code>Nilai</code> ≠ <code>nilai</code>	—
Tidak boleh ada spasi	<code>namaDepan</code>	<code>nama depan</code>

1.3 Scope (Ruang Lingkup) Variabel

```
int globalVar = 100; // GLOBAL — diakses di semua fungsi

int main() {
    int lokal = 10;   // LOKAL ke main()

    {
        int blok = 20; // LOKAL ke blok { }
        // lokal, globalVar, blok semuanya bisa diakses di sini
    }
    // blok sudah tidak bisa diakses di sini

    for (int i = 0; i < 5; i++) { // i hanya ada di dalam for
        cout << i;
    }
    // i sudah tidak bisa diakses di sini

    return 0;
}
```

```
}
```

■ **Best Practice:** Deklarasikan variabel sedekat mungkin dengan tempat penggunaannya. Hindari variabel global kecuali benar-benar diperlukan.

BAB 2 — Konstanta

2.1 const — Konstanta Bertipe

Keyword **const** membuat variabel yang nilainya tidak bisa diubah setelah inisialisasi. Compiler akan memberi error jika kode mencoba mengubahnya.

```
const double PI      = 3.14159265358979;
const int    MAX_NILAI = 100;
const string KOTA     = "Makassar";
const float  GRAVITASI = 9.81f;

// Penggunaan
double luas = PI * 5 * 5;    // 78.539...

// ERROR: tidak bisa mengubah nilai const
// PI = 3.14; <-- compile error!
```

2.2 #define — Makro Preprocessor

#define mengganti teks sebelum kompilasi (preprocessing). Tidak memiliki tipe data dan tidak menempati memori.

```
#define PI          3.14159
#define MAX         100
#define PESAN       "Halo Dunia"

// Makro fungsi (gunakan tanda kurung!)
#define KUADRAT(x) ((x) * (x))
#define MAKS(a,b)  ((a) > (b) ? (a) : (b))

int main() {
    cout << PI;           // compiler ganti PI → 3.14159
    cout << KUADRAT(5);    // 25
    cout << MAKS(3, 7);    // 7
}
```

2.3 constexpr — Konstanta Waktu Kompilasi (C++11)

constexpr menjamin nilai dievaluasi saat kompilasi, bukan saat runtime. Ini membuatnya lebih efisien dan lebih aman dari const.

```
constexpr double PI      = 3.14159265358979;
constexpr int    ARRAY_SZ = 50;
constexpr int    MAKS     = 100;

// Fungsi constexpr – dihitung saat kompilasi
constexpr int kuadrat(int n) { return n * n; }

int main() {
    constexpr int hasil = kuadrat(7); // 49, dihitung saat kompilasi
    int arr[ARRAY_SZ];               // ukuran array dari constexpr
    cout << hasil;
}
```

2.4 Perbandingan const vs #define vs constexpr

Aspek	const	#define	constexpr
Memiliki tipe data	■ Ya	■ Tidak	■ Ya
Dievaluasi saat	Runtime	Preprocessing	Compile-time
Berlaku dalam scope	■ Ya	■ Global	■ Ya
Mudah debug	■ Ya	■ Sulit	■ Ya
Bisa digunakan sbg ukuran array	■■ Tergantung	■ Ya	■ Ya
Rekomendasi penggunaan	■ Gunakan	■■ Hindari	■ Lebih baik

2.5 Enumerasi — enum & enum class

```
// enum biasa (C-style) — nilai mulai 0
enum Hari { SENIN, SELASA, RABU, KAMIS, JUMAT, SABTU, MINGGU };
//           0         1         2         3         4         5         6

// Nilai kustom
enum Status { AKTIF = 1, NONAKTIF = 0, PENDING = 99 };

// enum class (C++11) — aman, harus pakai scope
enum class Warna { MERAH, HIJAU, BIRU };
enum class Arah { UTARA, SELATAN, TIMUR, BARAT };

Hari    hari_ini = RABU;           // 2
Warna   cat      = Warna::MERAH;

if (cat == Warna::MERAH) cout << "Merah";
// if (cat == Arah::UTARA) — ERROR: tipe berbeda
```

■ **Tips:** Gunakan **enum class** (bukan enum biasa) di C++ modern karena lebih aman — mencegah konflik nama dan perbandingan antar enum berbeda.

BAB 3 — Operator

3.1 Operator Aritmatika

Digunakan untuk operasi matematika dasar.

Operator	Nama	Contoh	Hasil
+	Penjumlahan	5 + 3	8
-	Pengurangan	5 - 3	2
*	Perkalian	5 * 3	15
/	Pembagian	10 / 3	3 (int) / 3.33 (float)
%	Modulo (sisar)	10 % 3	1
++	Increment	x++	x = x + 1 (post)
--	Decrement	--x	x = x - 1 (pre)

```
int a = 10, b = 3;
cout << a + b;    // 13
cout << a - b;    // 7
cout << a * b;    // 30
cout << a / b;    // 3 (integer division, sisa diuang)
cout << a % b;    // 1 (sisa bagi)

// Perbedaan pre dan post increment
int x = 5;
cout << x++;    // 5 (tampil dulu, baru tambah)
cout << ++x;    // 7 (tambah dulu, baru tampil)

// Pembagian benar dengan float
cout << (double)a / b;    // 3.333...
```

3.2 Operator Assignment (Penugasan)

Operator	Contoh	Setara dengan	Keterangan
=	x = 5	—	Penugasan dasar
+=	x += 3	x = x + 3	Tambah & simpan
-=	x -= 3	x = x - 3	Kurang & simpan
*=	x *= 3	x = x * 3	Kali & simpan
/=	x /= 3	x = x / 3	Bagi & simpan
%=	x %= 3	x = x % 3	Modulo & simpan

```
int x = 10;
x += 5;    // x = 15
x -= 3;    // x = 12
x *= 2;    // x = 24
```

```
x /= 4;    // x = 6  
x %= 4;    // x = 2
```

■ **Good Practice:** Gunakan operator assignment gabungan (+=, -=, dll.) untuk kode yang lebih ringkas dan mudah dibaca.

3.3 Operator Relasional (Perbandingan)

Operator relasional membandingkan dua nilai dan menghasilkan **bool** (true atau false).

Operator	Nama	Contoh	Hasil
<code>==</code>	Sama dengan	<code>5 == 5</code>	true
<code>!=</code>	Tidak sama dengan	<code>5 != 3</code>	true
<code>></code>	Lebih besar	<code>5 > 3</code>	true
<code><</code>	Lebih kecil	<code>5 < 3</code>	false
<code>>=</code>	Lebih besar/sama	<code>5 >= 5</code>	true
<code><=</code>	Lebih kecil/sama	<code>3 <= 5</code>	true

```
int nilai = 78;
bool lulus = (nilai >= 70); // true
bool sempurna = (nilai == 100); // false

// PERHATIAN: = vs ==
// if (x = 5) // SALAH: ini assignment, bukan perbandingan
// if (x == 5) // BENAR: ini perbandingan
```

3.4 Operator Logika

Operator logika menggabungkan atau membalik ekspresi boolean.

Operator	Nama	Deskripsi	Contoh	Hasil
<code>&&</code>	AND (Dan)	true jika KEDUA kondisi true	<code>a>0 && b>0</code>	true jika a,b positif
<code> </code>	OR (Atau)	true jika SALAH SATU true	<code>a>0 b>0</code>	true jika minimal 1 positif
<code>!</code>	NOT (Tidak)	membalik nilai boolean	<code>!lulus</code>	true jika lulus=false

Tabel Kebenaran (Truth Table):

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'A && B' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica a-Bold', fontSize=10, greek=0, italic=0, link=[], rise=0, text='A ', textColor= Color(1,1,1,1), us_lines=[]), ParaF rag(__tag__='b', bold=1, fontName=' Helvetica-Bold', fontSize=10, greek=0, italic=0, link=[], rise=0, text='&', textColor= Color(1,1,1,1), us_lines=[]), ParaF rag(__tag__='b', bold=1, fontName=' Helvetica-Bold', fontSize=10, greek=0, italic=0, link=[], rise=0, text='B', textColor= Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0))#Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'B' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-B old', fontSize=10, greek=0, italic=0, link=[], rise=0, text='B', textColor=Color (1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0))#Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'A && B' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica a-Bold', fontSize=10, greek=0, italic=0, link=[], rise=0, text='A ', textColor= Color(1,1,1,1), us_lines=[]), ParaF rag(__tag__='b', bold=1, fontName=' Helvetica-Bold', fontSize=10, greek=0, italic=0, link=[], rise=0, text='&', textColor= Color(1,1,1,1), us_lines=[]), ParaF rag(__tag__='b', bold=1, fontName=' Helvetica-Bold', fontSize=10, greek=0, italic=0, link=[], rise=0, text='B', textColor= Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0))#Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'A B' 'frags': [ParaFrag(__tag__ ='b', bold=1, fontNa me='Helvetica-Bold ' , fontSize=10, greek=0, italic=0, link=[], rise=0, text='A B', textCol or=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0))#Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': '!A' 'frags': [ParaFrag(__tag__='b', bold=1, fo ntName='Helvetica-Bo ld', fontSize=10, greek=0, italic=0, link=[], rise=0, text='!A', textColor=Co lor(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0))#Paragraph
true	true	true	true	false
true	false	false	true	false
false	true	false	true	true
false	false	false	false	true

```
int nilai = 85;
bool hadir = true;

// Gabungan operator logika
if (nilai >= 70 && hadir == true)
    cout << "Lulus";
```

```
if (nilai < 50 || hadir == false)
    cout << "Tidak Lulus";

// Short-circuit evaluation
// && : jika kiri false, kanan tidak dievaluasi
// || : jika kiri true, kanan tidak dievaluasi
if (ptr != nullptr && *ptr > 0)    // aman: cek null dulu
    cout << *ptr;
```

■■ **Perhatian:** Jangan keliru antara && (logika) dan & (bitwise), serta || (logika) dan | (bitwise). Keduanya berbeda cara kerjanya!

3.5 Operator Bitwise

Operator bitwise bekerja langsung pada bit-bit bilangan integer. Sering digunakan dalam pemrograman sistem, embedded, dan manipulasi flag.

Operator	Nama	Contoh (desimal)	Keterangan
&	AND bitwise	12 & 10 = 8	Bit 1 jika KEDUA bit 1
	OR bitwise	12 10 = 14	Bit 1 jika SALAH SATU bit 1
^	XOR bitwise	12 ^ 10 = 6	Bit 1 jika bit BERBEDA
~	NOT bitwise	~12 = -13	Balik semua bit
<<	Shift kiri	5 << 1 = 10	Geser bit ke kiri ($\times 2$)
>>	Shift kanan	10 >> 1 = 5	Geser bit ke kanan ($\div 2$)

```
// Visualisasi bitwise
// 12 = 1100 (biner)
// 10 = 1010 (biner)

cout << (12 & 10);    // 8 = 1000
cout << (12 | 10);    // 14 = 1110
cout << (12 ^ 10);    // 6 = 0110
cout << (~12);        // -13 (komplemen dua)

// Shift: cara cepat perkalian/pembagian dengan 2
int x = 5;
cout << (x << 1);    // 10 (5  $\times$  2)
cout << (x << 2);    // 20 (5  $\times$  4)
cout << (x >> 1);    // 2 (5  $\div$  2, dibulatkan ke bawah)

// Aplikasi: flag/bit masking
const int FLAG_READ  = 0b001;    // 1
const int FLAG_WRITE = 0b010;    // 2
const int FLAG_EXEC  = 0b100;    // 4

int izin = FLAG_READ | FLAG_WRITE;    // 0b011 = 3
if (izin & FLAG_READ) cout << "Bisa baca";
if (izin & FLAG_WRITE) cout << "Bisa tulis";
if (izin & FLAG_EXEC) cout << "Bisa eksekusi";    // tidak muncul
```

3.6 Operator Ternary & sizeof

```
// Operator Ternary: kondisi ? nilai_true : nilai_false
int nilai = 80;
string ket = (nilai >= 70) ? "Lulus" : "Tidak Lulus";
cout << ket;    // Lulus

// Bisa bersarang (tapi jangan terlalu dalam)
string grade = (nilai >= 90) ? "A" :
               (nilai >= 80) ? "B" :
               (nilai >= 70) ? "C" : "D";

// sizeof – ukuran tipe data dalam byte
cout << sizeof(int);    // 4
cout << sizeof(double);    // 8
```

```
cout << sizeof(char);          // 1
cout << sizeof(long long);    // 8

int arr[10];
int jumlah_elemen = sizeof(arr) / sizeof(arr[0]); // 10
```

3.7 Prioritas Operator (Operator Precedence)

Urutan prioritas menentukan operator mana yang dievaluasi terlebih dahulu. Operator dengan prioritas lebih tinggi dieksekusi lebih dahulu.

Prioritas	Operator	Nama	Asosiatif
1 (tertinggi)	() [] .	Kurung, subscript	Kiri ke kanan
2	++ -- ! ~	Unary (prefix)	Kanan ke kiri
3	* / %	Perkalian, bagi, modulo	Kiri ke kanan
4	+ -	Penjumlahan, pengurangan	Kiri ke kanan
5	<< >>	Shift bitwise	Kiri ke kanan
6	< <= > >=	Relasional	Kiri ke kanan
7	== !=	Kesamaan	Kiri ke kanan
8	&	AND bitwise	Kiri ke kanan
9	^	XOR bitwise	Kiri ke kanan
10	 	OR bitwise	Kiri ke kanan
11	&&	AND logika	Kiri ke kanan
12	 	OR logika	Kiri ke kanan
13	?:	Ternary	Kanan ke kiri
14 (terendah)	= += -= ...	Assignment	Kanan ke kiri

```
// Contoh prioritas
int a = 2 + 3 * 4;          // 14, bukan 20 (* lebih tinggi dari +)
int b = (2 + 3) * 4;       // 20 (kurung ubah prioritas)

bool c = 5 > 3 && 2 < 4;    // true (> dan < dulu, baru &&)
bool d = true || false && false; // true (&& dulu, baru ||)

// Gunakan kurung untuk kejelasan!
bool e = (true || false) && false; // false
```

BAB 4 — Contoh Program Lengkap

Program: Kalkulator BMI dengan Klasifikasi

Program ini menggabungkan variabel, konstanta, enum, dan berbagai operator dalam satu aplikasi nyata.

[illegible]

```

        hitungan++;

        cout << nama << ", BMI Anda: " << bmi << endl;
        cout << "Kategori   : " << getKategori(bmi) << endl;
        cout << "Saran       : " << saran << endl;

        cout << "\nHitung lagi? (1=Ya/0=Tidak): ";
        cin >> lanjut;

    } while (lanjut);

    cout << "Total perhitungan: " << hitungan;
    return 0;
}

```

■ Ringkasan Materi

Topik	Keyword / Sintaks	Catatan Penting
Variabel	type nama = nilai;	Selalu inisialisasi
Scope lokal	{ } blok kode	Variabel hanya ada di { }
const	const type NAMA = nilai;	Tidak bisa diubah
constexpr	constexpr type NAMA = nilai;	Dievaluasi saat kompilasi
#define	#define NAMA nilai	Hindari — gunakan const
enum class	enum class N { A, B };	Akses dengan N::A
Aritmatika	+ - * / % ++ --	Hati-hati integer division
Assignment	+= -= *= /= %=	Lebih ringkas
Relasional	== != > < >= <=	Jangan campur = dan ==
Logika	&& !	Short-circuit evaluation
Bitwise	& ^ ~ << >>	Bekerja pada level bit
Ternary	cond ? val_t : val_f	Pengganti if-else singkat
sizeof	sizeof(type)	Ukuran dalam byte